

Programmeringsparadigmer 2025

Kursusgang 7: Erklæring af typer og typeklasser

Hans Hüttel

22. oktober 2025

Vores plan for i dag

1. Læringsmål
2. Præsentation af forberedelsesopgaverne
3. Opgave: Udtryksformer
4. Opgave: Evaluering af udtryk
5. Pause
6. Diskussion
7. Opgave: Encyklopædier
8. Opgave: Lagdelte encyklopædier
9. Opgave: Skab (vektor)rum for dig selv
10. Der er også altid ”bogstavopgaver”, man kan kaste sig over!
11. Vi evaluerer dagens kursusgang – **bliv venligst til slut!**

Læringsmål

Den store morale: Man kan lave sine egne typer, typeklasser og instanser af typeklasser i Haskell.

- ▶ At forstå, hvordan type, newtype og data-erklæring fungerer, og hvordan de adskiller sig hinanden.
- ▶ At kunne definere funktioner over rekursivt definerede datatyper ved hjælp af mønstermatch.
- ▶ At forstå, hvordan rekursivt definerede datatyper kan bruges til at implementere dannelsesregler for et sprog.
- ▶ At forstå begrebet termkonstruktører og, hvordan de bruges.
- ▶ At forstå principperne for og kunne deklarere nye instanser af typeklasser.
- ▶ At forstå, hvordan ovenstående begreber kan anvendes i en større programsammenhæng.

Forberedelsesopgave – Unære tal

Unære tal består af en endelig sekvens af I'er efterfulgt af et Z. Det naturlige tal n kan repræsenteres som n successive I'er og et Z, så f.eks. 4 repræsenteres i unær notation som IIIIZ. Det naturlige tal 0 repræsenteres som Z.

Definér en rekursiv datatype `Unary` for unære tal og brug din type-definition til at definere en funktion `unary2int` af typen `unary2int :: Unary -> Integer`, der finder det naturlige tal, som et givet tal repræsenterer. Som eksempel, skal `unary2int (I I I I Z)` give os 4.

Forberedelsesopgave – least

Brug erklæringen af typen `Tree` på side 97 til at definere en funktion `least`, der finder det mindste element i et givet binært træ. Hvad skal typen af `least` være?

De næste to uger

I næste uge samler vi op – og besøger opgaver, vi ikke nåede, og opgaver om det, der kan være brug for at besøge én gang til.

Om to uger er der nyt stof igen og forberedelsesopgaver. Hvem kunne tænke sig at fortælle om hvad de har lavet?

Opgave 1 – Aritmetiske udtryk (10 minutter)

Definér en Haskell-datatype `Aexp` for aritmetiske udtryk med addition, multiplikation, tal og variable. Dannelsesreglerne er

$$E ::= n \mid x \mid E_1 + E_2 \mid E_1 \cdot E_2$$

Antag, at variable x er strenge, og at tal n er heltal.

Opgave 2 – Evaluering af udtryk (20 minutter)

Brug din Haskell-datatype fra den foregående opgave til at definere en funktion `eval`, som, når den får en term af typen `Aexp` og en tildeling `ass` af variable til tal, kan beregne værdien af udtrykket.

Et eksempel: Hvis vi har tildelingen $[x \mapsto 3, y \mapsto 4]$, skal `eval` kunne fortælle os, at værdien af $2 \cdot x + y$ er 10.

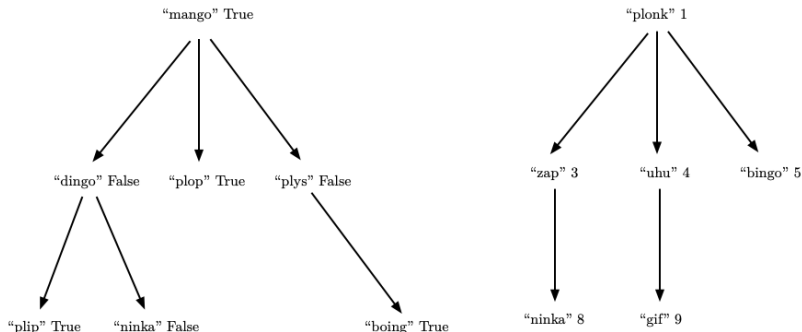
Hint: En tildeling er en funktion. Hvordan kan du repræsentere den?

Pause

Diskussion – En encyklopædi

Forfatterne til en ny encyklopædi om værdier af typen **a** repræsenterer deres information ved hjælp af en hierarkisk struktur, hvor hver post i encyklopædien er mærket med en nøgle, der er en streng, og en værdi af typen **a**. Poster kan have underposter.

Her er to encyklopædier (vist også i opgavesættet).



Diskussion – En encyklopædi

Forfatterne til encyklopædien bad en berømt influencer om at definere en type `Encyclopedia` for encyklopædier. Influenceren skrev:

```
data Encyclopedia a = Leaf String a |  
    Node1 String a (Encyclopedia a) | Node2  
    String a (Encyclopedia a) (  
    Encyclopedia a) | Node3 String a (  
    Encyclopedia a) (Encyclopedia a) (  
    Encyclopedia a)  
deriving Show
```

men klagede over Haskell's begrænsninger: *“Jeg kan se af eksemplerne, at enhver post højst kan have tre underposter. Intet i Haskell tillader at skrive, at der kan være vilkårligt mange børn af samme type, så jeg er nødt til at skrive en meget klodset definition. **Haskell er ynkeligt!**”*

Opgave 3 – En bedre løsning (20 minutter)

- ▶ Kom med en bedre definition af typen **Encyclopedia**. Find ud af, hvad der er galt, og kom med en bedre løsning. Det er en god idé at læse opgaveteksten meget omhyggeligt og finde ud af, hvad den siger og ikke siger. Når du har kritiseret den eksisterende løsning, er det også en god idé **ikke at forsøge at reparere** den eksisterende løsning, men at starte forfra.
- ▶ Vis, hvordan du repræsenterer t_1 og t_2 fra tidligere som værdier **t1** og **t2** af din type.

Opgave 4 – Lagdelte encyklopædier (20 minutter)

En encyklopædi er *lagdelt*, hvis det gælder, at alle værdier på samme niveau i encyklopædien er større end værdierne på niveauerne ovenover. Som eksempel er t_2 fra tidligere lagdelt, da 8 og 9 på niveau 3 er større end værdierne 3, 4 og 5 på niveau 2 – som igen er større end værdien 1 på niveau 1.

Definér en funktion `layered`, der kan fortælle os, om en encyklopædi er lagdelt.

Opgave 5 – Definér din egen typeklasse (20 minutter)

Et vektorrum med indre produkt er et rum, for hvilket operationerne vektorsum og prikprodukt er defineret.

Givet to vektorer v_1 og v_2 , er summen $v_1 + v_2$ igen en vektor, og det indre produkt $v_1 \cdot v_2$ er et tal. *Vær opmærksom på:* I lineær algebra kræver definitionen af indre produktrum mere end dette! Antag, at det indre produkt er et tal af typen `Int`.

Definér en typeklasse `InVector`, hvis instanser er typer, der kan betragtes som indre produktrum, hvor vektorsum kaldes `&&&` og indre produkt kaldes `***`.

Find ud af, hvordan du deklarerer `Bool` som en instans af `InVector`.

Evaluering

- ▶ Hvad fandt I svært?
- ▶ Hvad overraskede jer?
- ▶ Hvad gik godt?